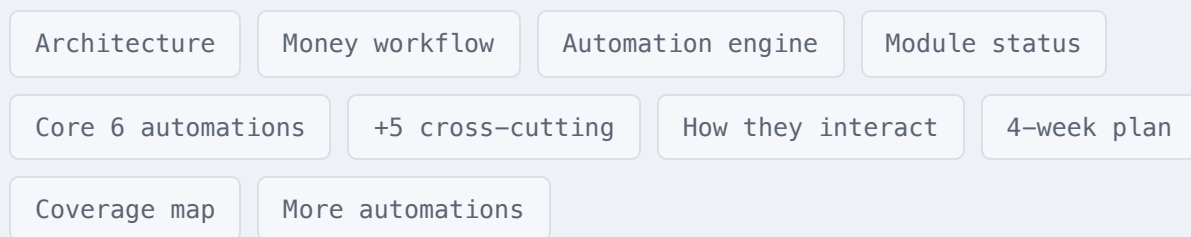


ERP: what's built, how money moves, and 11 automations for the next 4 weeks

A working map of the current platform — real estate core, multi-business kernel, and the automation engine that already runs the Invoice (AP) automation — plus 11 new automations (Communication, Receivables, Expenses, Leads/CRM, HR, and five cross-cutting wins) sequenced into a 4-week build plan, and what else the platform needs to genuinely handle any business.



01 — PLATFORM SHAPE

A shared kernel, thin verticals, one automation engine

Real Estate is the deep, hand-built vertical. Everything else — Construction, Education's sales side — runs on a generic Item + Sale engine that posts to the same ledger. The automation platform sits on top of all of it and is where every new capability plugs in.

VERTICAL MODULES – deep, hand-built

Real Estate
Plots · Bookings · Installments

Education
Students · Attendance

Generic engine
Item + Sale



KERNEL – built once, shared by every business

Tenancy & Business Units

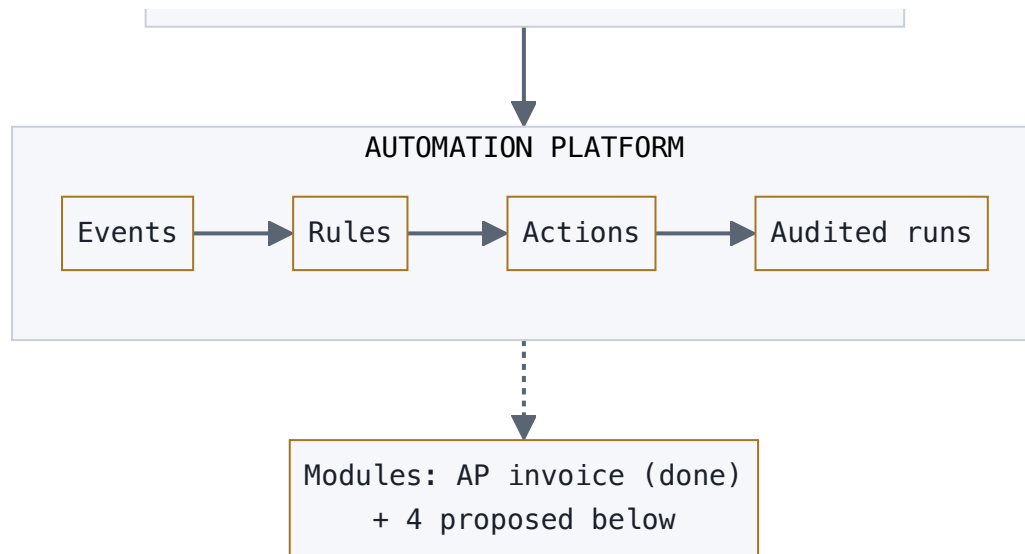
Users / RBAC

Customers

Ledger – source of truth

Payments

REST API

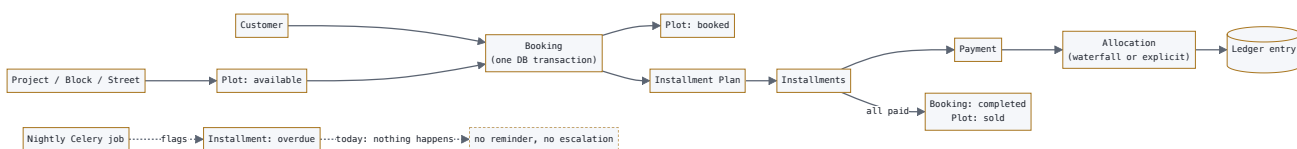


Adding automation #51 is a new module, not a rewrite of the boxes above it.

02 – CORE MONEY WORKFLOW

Booking → installments → payment → ledger

One transaction creates the booking, flips the plot, opens the installment plan, and posts the receivable. Corrections are reversing entries, never edits — this is what makes the ledger trustworthy as the one source of truth.

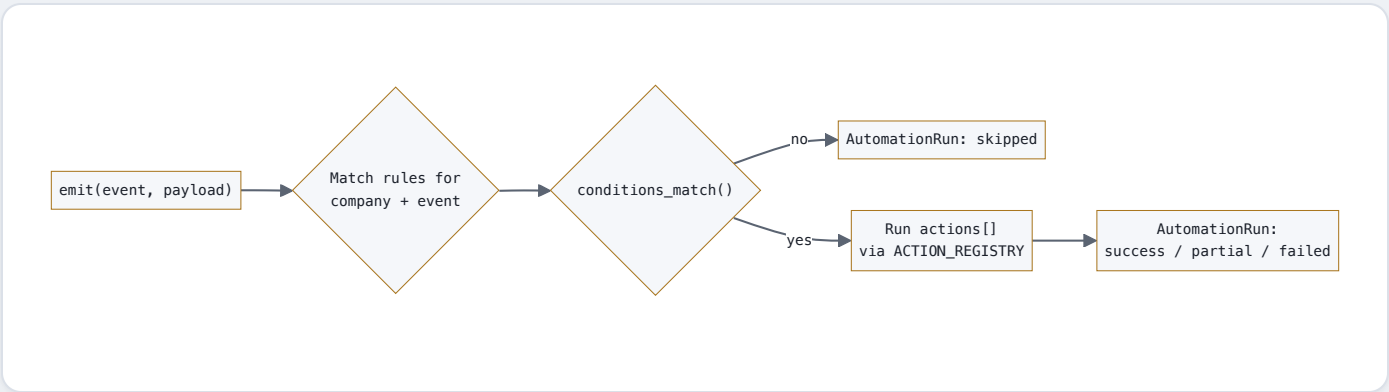


The dashed path is the actual gap Automation #2 (below) closes.

03 – THE AUTOMATION ENGINE

Events → rules → actions → audited runs

Services call `emit()` and never know who's listening. Rules are data (JSON conditions + JSON action list) so admins configure automations without a deploy. Every dispatch is recorded, success or failure.



Events firing in production today (grep-verified):

EVENT	SOURCE
booking.created	bookings/services.py
payment.recorded	payments/services.py
sale.created	businesses/services.py
attendance.recorded / attendance.absent	education/views.py
ap_invoice.received / ap_invoice.processed	accounts_payable/services.py
record.created	studio/views.py

04 – MODULE STATUS

What's real vs. what's a shell

Condensed from the full feature table — this is the shape of the gap the roadmap targets.

Real Estate core

done

Plots, bookings, installments, payments, ledger — tested, transactional, reversible.

Multi-business kernel

done

Business units, generic Item/Sale, consolidated + per-division dashboards.

Automation engine

done

Event bus, rules, actions, audited runs — production-grade, thin surface so far.

AP invoice automation

done

Email ingest, OCR, PO matching, anomaly detection, approvals, QuickBooks sync.

Education

done

Students, teachers, attendance — second hand-built vertical, already emits events.

Reports

done

Collections, sales, aging, cash-flow — all scoped + date-ranged. No P&L yet.

Customer invoices

frontend only

Full UI built; `invoiceService.js` runs on mock data — no backend model.

Expenses

partial

CRUD + status field only. No approval workflow, no ledger posting, no events.

CRM

frontend only

`Crm.vue`, 461 lines — no API calls, no backend at all.

HR / Employees

frontend only

Shift management UI exists; no backend models found.

WhatsApp

disconnected

Standalone Node/Mongo service runs independently — not wired to the action registry.

Documents / DMS

model only

`CustomerDocument` exists; no upload UI, no e-signature.

05 — PROPOSED, CORE

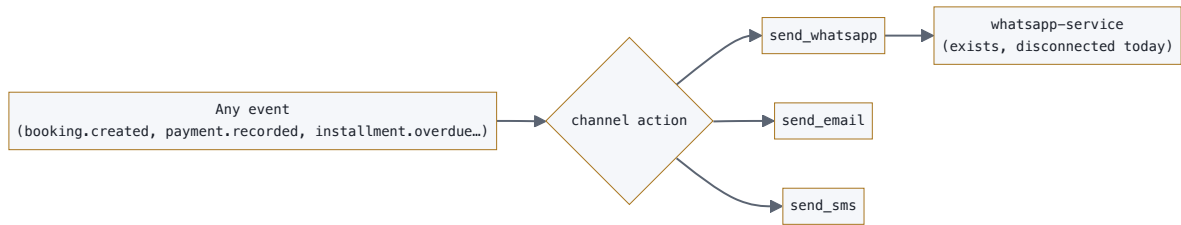
Six core automations, ordered by dependency

Each follows the exact shape the AP invoice module already proved out: an

`automation_module.py`, new `@register_action` handlers, new `emit()` call sites.

01 Omnichannel notification channel foundational

Every automation below wants to tell someone something. Today the only real delivery is an in-app notice or a raw webhook — despite a WhatsApp micro-service already running, disconnected. Build this first; it's what turns "notify" into "reach the person."



NEW

DETAIL

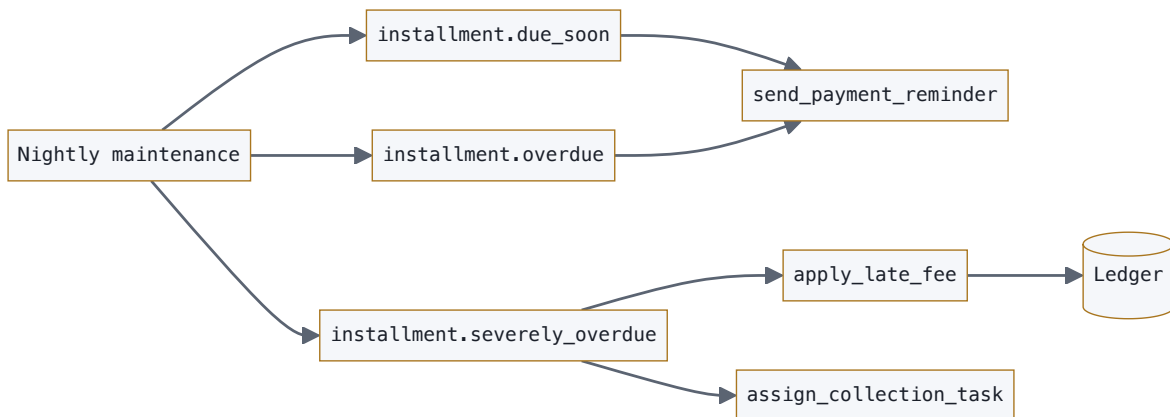
Actions `send_whatsapp`, `send_email`, `send_sms` — template-rendered via the existing `render()` helper

Module `communication/automation_module.py`, key `omnichannel`

Data model None required for v1 (stateless dispatch)

02 Receivable collections & dunning highest ROI

The AP side just got a deep automation; the AR side — the installments the whole Real Estate vertical is built around — has none. The nightly job flags overdue installments and tells nobody. The late-fee calculation already exists; only "post it" is missing.



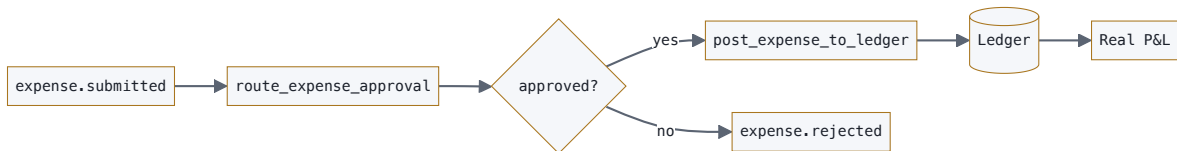
NEW

DETAIL

Events	<code>installment.due_soon</code> , <code>installment.overdue</code> , <code>installment.severely_overdue</code> , <code>late_fee.applied</code>
Actions	<code>send_payment_reminder</code> , <code>apply_late_fee</code> , <code>assign_collection_task</code> , <code>flag_customer_risk</code>
Data model	Optional <code>Customer.risk_tier</code> field only

03 Expense approval & ledger posting unlocks P&L

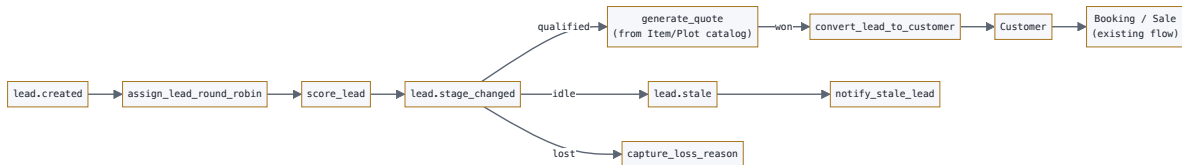
Expenses are a disconnected log today — no workflow, no ledger entry. This is the single blocker to a real P&L, and the natural second "approval" automation after AP invoices, reusing the same routing pattern.



NEW	DETAIL
Events	<code>expense.submitted</code> , <code>expense.approved</code> , <code>expense.rejected</code>
Actions	<code>route_expense_approval</code> , <code>post_expense_to_ledger</code> , <code>check_budget_threshold</code>
Data model	<code>Expense.approved_by/at</code> + a small <code>ExpenseApprovalPolicy</code> reference row

04 Leads & CRM — lead-to-cash biggest new surface

The CRM UI is fully designed with zero backend — the largest gap between promise and reality in the codebase. It's also the only automation here that covers the funnel *before* a customer exists: scoring, quoting, and nurturing, feeding the same Customer → Booking/Sale flow rather than forking it.



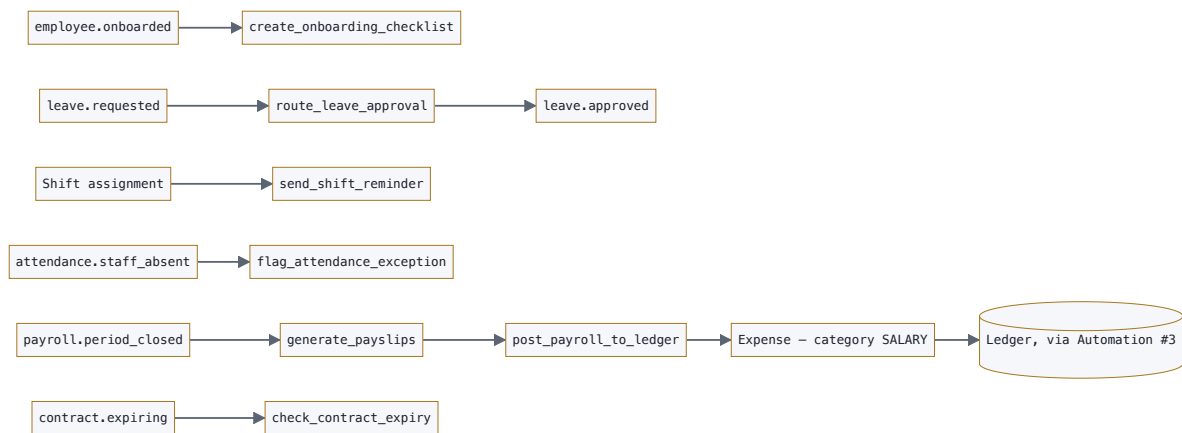
NEW

DETAIL

App	crm/ — new Django app, <code>Lead</code> + <code>Quote</code> , no parallel customer/pricing model
Events	<code>lead.created</code> , <code>lead.scored</code> , <code>lead.stage_changed</code> , <code>lead.won</code> , <code>lead.lost</code> , <code>lead.stale</code>
Actions	<code>assign_lead_round_robin</code> , <code>score_lead</code> , <code>generate_quote</code> , <code>convert_lead_to_customer</code> , <code>create_followup_task</code> , <code>notify_stale_lead</code> , <code>capture_loss_reason</code>
Nurture cadence	A "3-touch follow-up" is just three rules with a delay condition — no special sequence engine needed

05 HR — onboarding, leave, attendance & payroll kernel-level

HR (`HR.vue`, `Employees.vue` — dashboard, employees, payroll, leave, shifts) is, like CRM, a fully-designed frontend running on hard-coded arrays. Unlike CRM, it isn't vertical-specific — every business unit has staff — so it belongs in the shared kernel, next to Customers. Payroll deliberately doesn't invent its own ledger-posting path: it creates an `Expense(category=SALARY)` row and rides Automation #3's existing pipeline to the ledger.



NEW	DETAIL
App	hr/ — kernel-level, not a vertical module: <code>Employee</code> , <code>LeaveRequest</code> , <code>ShiftAssignment</code> , <code>PayrollRun</code> , <code>Payslip</code>
Events	<code>employee.onboarded</code> , <code>leave.requested/approved/rejected</code> , <code>attendance.staff_absent</code> , <code>contract.expiring</code> , <code>payroll.period_closed</code> , <code>employee.offboarded</code>
Actions	<code>create_onboarding_checklist</code> , <code>route_leave_approval</code> , <code>flag_attendance_exception</code> , <code>send_shift_reminder</code> , <code>generate_payslips</code> , <code>post_payroll_to_ledger</code> , <code>check_contract_expiry</code> , <code>create_offboarding_checklist</code>
Composition	Payroll → Expense → Ledger reuses Automation #3 rather than posting money twice

Two primitives are worth building once in the automation kernel, in Week 1: a shared `Task` (`assignee` , `title` , `due_date` , `status`) — Collections (#2), Leads (#4), and HR (#5) all want the exact same "assign a follow-up" shape — and a shared `ApprovalRequest` , since AP's existing approval, Expenses (#3), and HR Leave (#5) would otherwise each reinvent "something needs sign-off from someone."

Smaller than #1–#6 — most are a day or less — precisely because each leans on a primitive the core six already build.

#7 Sales commission

new

`booking.completed` →
`calculate_commission` → routed through #3's expense/ledger pipeline. No second payout mechanism — a commission is just another Expense.

#8 Cross-sell / 360° fit

new

Uses the platform's real differentiator — one shared Customer across every Business Unit — to flag a plot buyer who's never been offered Education, or vice versa.

#9 Duplicate / data quality

new

`lead.created` / `customer.created` →
`check_duplicate` on phone/email/ID → notify with a merge suggestion instead of a silent second record.

#10 Approval delegation on leave

new

Combines #5 and the shared `ApprovalRequest`: when an approver goes on leave, their pending AP/expense/leave approvals auto-reassign to a backup.

#11 Scheduled report delivery

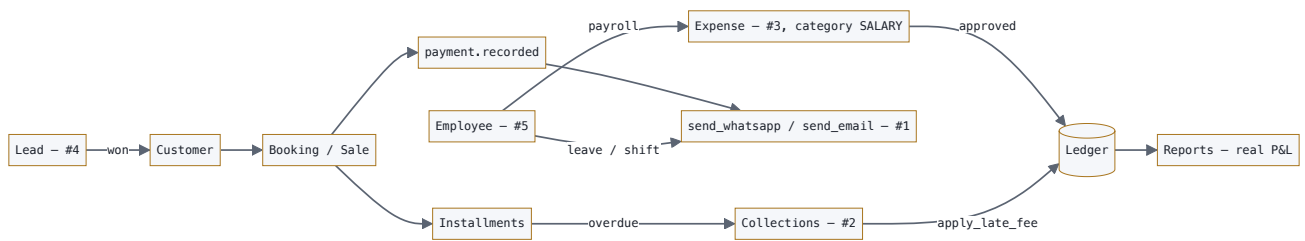
new

A weekly Celery-beat tick → `send_email` with the numbers, PDF once report export ships. Turns "we have reports" into "the owner gets Monday's numbers unprompted."

07 — PUT TOGETHER

How the six connect end to end

None of this needs a new framework — every arrow below runs on the same event bus already carrying AP invoices in production.

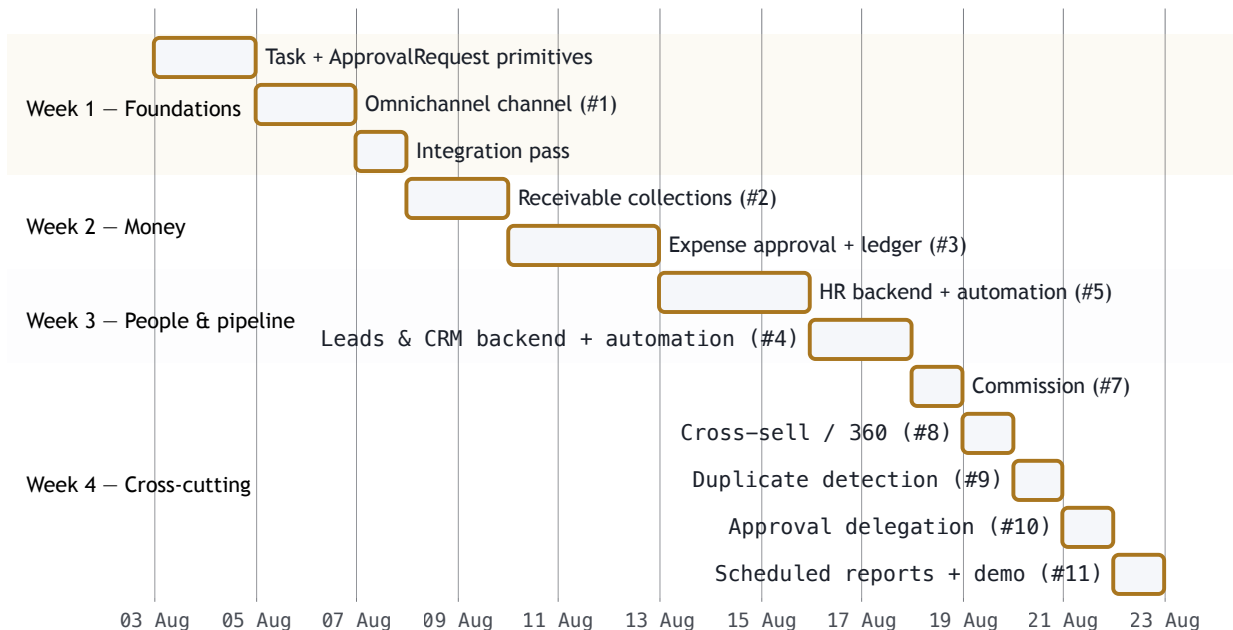


08 – SEQUENCING

The 4-week build plan

Ordered by dependency, not just value — Week 1 is plumbing everything else needs, even though it's the least glamorous to demo.

11 automations across 4 weeks



Task + ApprovalRequest primitives, then the send_whatsapp/email/sms actions. Unblocks every reminder, approval, and confirmation below it.

2 **Receivables (#2) → Expenses (#3)**

Highest immediate cash-flow return, then the approval + ledger posting that unlocks a real P&L.

3 **HR (#5) → Leads & CRM (#4)**

The two biggest net-new surface areas. HR first since payroll leans on Week 2's expense/ledger pipeline.

4 **#7–#11, one per day**

Commission, cross-sell, duplicate detection, approval delegation, scheduled reports — each a thin layer on what Weeks 1–3 already built.

09 – BECOMING TRULY GENERAL

What else a "handle any business" ERP needs

The single highest-leverage gap is the **no-code metadata layer** — custom entities, fields, and pipelines — already named as Phase 4 in `platform-vision.md` but never built. It matters more than any one module below because it turns "cover a new industry" from a development project into a configuration task.

No-code custom entities

strategic

Define a new record type (Vehicle, Patient, Case) with fields + pipeline, no migration. Plugs into the existing event bus automatically.

Inventory / stock

missing

`Item` has no stock-quantity concept yet. Small, additive — unlocks Procurement and POS behind it.

Internal task / project tracking

missing

Every business needs this regardless of vertical. The shared `Task` primitive above is most of the data model already.

Procurement

missing

PO creation and vendor selection *before* a bill arrives — AP only handles paying it after.

Helpdesk / Support

missing

Subscription / recurring billing

missing

Same assignment/SLA/escalation shape as Leads and Collections — a natural fit once the metadata layer exists.

Posts to the same ledger as everything else — valuable for any membership or retainer-based division.

POS / Retail

missing

Explicit Phase 5 in the vision doc; depends on Inventory existing first.

Bank reconciliation

missing

Ledger is solid, but nothing matches it against an imported bank statement yet.

Vendor management

partial

AP processes a vendor's bill; no onboarding, scorecard, or preferred-vendor list beyond that.

Appointment / scheduling

missing

Site visits, service calls, consultations — any vertical with a calendar-based touchpoint.

Customer / agent portals

missing

No external login for a customer's own statement, or an agent's own pipeline.

Report export (PDF/Excel)

missing

Flagged  in FEATURES.md; blocks Automation #11's scheduled reports.

Audit trail

partial

Fields exist on models; no API population, no browsable change log.

i18n / RTL, SSO / 2FA

missing

Currency/tax/units are configurable; UI strings and auth security are not yet.

10 — MENU

More automation ideas, by area

Beyond the six specced above — each roughly a trigger → action pair once its owning module exists.

AREA	TRIGGER	ACTION(S)
Sales	<code>quote.sent</code> + no reply	send_whatsapp follow-up
Sales	<code>customer.birthday</code>	send_whatsapp greeting
Sales	<code>referral.customer_converted</code>	calculate_referral_commission

Finance	<code>bank_statement.imported</code>	<code>match_to_ledger</code> , <code>flag_unmatched</code>
Finance	<code>fx_rate.stale</code>	<code>fetch_fx_rate</code> , <code>notify</code>
HR	<code>employee.probation_ending</code>	<code>create_followup_task</code> , <code>notify manager</code>
HR	<code>timesheet.missing</code>	<code>notify employee + manager</code>
Inventory	<code>stock.low</code>	<code>notify</code> , <code>create_purchase_order</code>
Support	<code>ticket.sla_breach_imminent</code>	<code>escalate</code> , <code>notify manager</code>
Compliance	<code>identity_doc.expiring</code>	<code>send_whatsapp/email reminder</code>
Compliance	<code>contract.renewal_due</code>	<code>create_followup_task</code> , <code>notify</code>
Platform	<code>automation.run_failed</code>	<code>notify admin</code> — automations watching automations
Education	<code>attendance.absent</code> (<i>already emits</i>)	<code>notify_guardian</code> — only the action is missing

Full detail and data-model diffs live in the repo's `docs/` folder:

`docs/erp-overview.md`

`docs/automation-roadmap.md`

`docs/4-week-execution-plan.md`

`docs/erp-module-coverage.md`

`docs/FEATURES.md`